

A Comparison of Heuristic and Metaheuristic Methods for solving the Traveling Salesman Problem

1st Daniel Schafh  tle
Computational and Data Science
FHGR
Chur, Switzerland
daniel.schafhaute@stud.fhgr.ch

2nd Joshua Kohler
Computational and Data Science
FHGR
Chur, Switzerland
joshua.kohler@stud.fhgr.ch

3rd Gian-Luca Lanfranchi
Computational and Data Science
FHGR
Chur, Switzerland
gianluca.lanfranchi@stud.fhgr.ch

Abstract—The traveling salesman problem (TSP) is an NP hard combinatorial optimization problem, that asks for the shortest possible route that visits a set of cities and returns to the starting city. Solving TSPs exactly using brute-force methods quickly becomes computationally infeasible as the number of cities increases. However, there exists a number of heuristic and metaheuristic algorithms, that can find reasonably good solutions in a much shorter time. This paper presents a comparison of five such methods, namely: Simulated Annealing, Tabu Search, 2-Opt, Iterated Local Search and a Genetic Algorithm. Algorithms are evaluated using synthetic benchmark TSP instances of sizes $n = [10, 25, 50, 100, 200, 500, 800, 1000, 1500, 2000]$ with $k = 30$ instances each. We perform this experiment with two types of instances: one with uniformly sampled cities and one with clustered cities. They are then compared based on the solution quality measured as the gap to the optimal solution obtained by the Concorde TSP solver, and the time required to find the solution. The goal of this paper is to compare the strengths and weaknesses of these algorithms, that are the basis of many state of the art methods for solving TSPs and provide a starting point for further research into this area.

I. INTRODUCTION

The traveling salesman problem (TSP) is a classic combinatorial optimization problem that asks for the shortest possible route that visits a set of cities and returns to the origin city. More formally, given a complete Graph $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_n\}$ is the set of nodes representing the cities, and $d(v_i, v_j)$ is the distance associated with the edge between each pair of nodes $v_i, v_j \in V$, then the solution to a TSP is the ordering of cities $(v_{\pi(1)}, v_{\pi(2)}, \dots, v_{\pi(n)})$ that minimizes the total tour length:

$$\left(\sum_{i=1}^{n-1} d(v_{\pi(i)}, v_{\pi(i+1)}) \right) + d(v_{\pi(n)}, v_{\pi(1)}) \quad (1)$$

Where $\pi(i)$ determines the index of the city that is visited at step i [1].

This is an NP-hard combinatorial optimization problem. That means, that there is no known polynomial time algorithm, that can solve it optimally for all instances [1]. It is so hard to solve, because the number of possible tours is $(n - 1)!/2$

and grows with $\mathcal{O}(n!)$ [1]. This makes it infeasible to brute-force a solution for problems with more than a few nodes. A problem with 100 nodes has 10^{155} possible tours, to check all of them naively would take longer than the age of the universe. Because we don't have that kind of time, we need to find ways to quickly find solutions to TSPs that are *good enough*, meaning that they are not optimal, but sufficiently close to the optimal solution. However, for a lot of applications a solution that is not quite optimal is all that is needed. Heuristics are used to find these near optimal solutions within reasonable time.

II. RELATED WORK

The TSP is one of the most intensively studied combinatorial optimization problems. Its NP-hard nature [1] makes exact solution methods infeasible for large instances, yet the search for optimal solutions has driven significant progress in both exact and heuristic algorithms for solving the TSP but also in the area of combinatorial optimization as a whole.

A. Exact Solvers

Exact solutions to the TSP can be obtained by modelling it as a dynamic programming or integer linear programming problem and then solving it using branch-and-bound or branch-and-cut methods [2]. The gold standard for exact solvers of the TSP is Concorde [3], it has been developed and optimized over years specifically to solve TSPs and was able to obtain an optimal tour for a problem with 85900 cities, computing it required over 136 CPU years [3]. While exact solvers are the gold standard for verifying solution quality, their runtime increases exponentially, limiting their applicability to large instances.

B. Heuristics and Metaheuristics for the TSP

To overcome the limitations of exact methods, researchers have developed a wide range of heuristic and metaheuristic algorithms that can find near-optimal solutions in reasonable time.

Heuristics for the TSP can be separated into two groups of algorithms: *tour construction procedures*, which incrementally build a tour by adding a new node to the tour at each step, and *tour improvement procedures*, which start from an initial tour and then try to improve that tour by making changes. *Composite procedures* are algorithms, that start with a solution generated by a construction procedure and then improve it with an improvement procedure [4].

C. History and Developements

Among the most influential metaheuristics, *Simulated Annealing* (SA) introduced the idea of probabilistic acceptance of worsening moves to escape local optima.

Tabu Search (TS), originally proposed by Glover [4] and later refined by Gendreau and Potvin [5], uses explicit memory structures (tabu lists) to prevent cycling and guide the search.

Iterated Local Search (ILS) [6] builds chains of local optima by perturbing solutions between applications of a local search, achieving state of the art performance on many problems.

Genetic Algorithms (GAs) for the TSP are reviewed by Larrañaga et al. [7], and more recent developments, including the EAX crossover that can find optimal tours for instances with up to 200,000 cities, are discussed by Whitley [8].

A more in-depth review of metaheuristics and heuristics applied to TSP is provided by Gendreau and Potvin [9], where they present a wide variety of both basic and advanced algorithms.

III. MATERIALS AND METHODS

In this chapter several algorithms are introduced that are at the basis of many state of the art methods for solving the TSP. The goal is to introduce the reader to a variety of different techniques and concepts that are commonly used.

A. 2-Opt

2-Opt is one of the simplest and most widely used local search heuristics for the TSP [4]. Starting from an initial tour, it repeatedly removes two non-adjacent edges and reconnects the resulting two paths in the only other possible way, which is the same as reversing the segment between the two cut points. The tour is cut into two segments A, B , and then reconnected as $A, reversed(B)$. A move is accepted whenever it strictly decreases the tour length. The neighborhood of a tour with n cities therefore contains $\mathcal{O}(n^2)$ candidate moves. The search terminates in a local minimum, that is, a tour for which no single 2-Opt swap gives an improvement.

In this study, the initial tour is constructed by a greedy nearest-neighbor heuristic: starting from a fixed city, the next city is always chosen as the closest unvisited one until all cities are included. The 2-Opt search itself uses a *first-improvement* strategy: as soon as an improving swap (i, j) is found, it is applied immediately and the scan is restarted from the beginning of the tour. This is repeated until a full pass over all pairs of edges (i, j) finds no improving move, at which point the tour is 2-optimal [4].

Algorithm 1 2-Opt

```

1:  $s = \text{GreedyInitialTour}()$ 
2:  $improved \leftarrow \text{true}$ 
3: while  $improved$  do
4:    $improved \leftarrow \text{false}$ 
5:   for  $i = 0$  to  $n - 3$  do
6:     for  $j = i + 2$  to  $n - 1$  do
7:        $a, b \leftarrow s_i, s_{i+1}$ 
8:        $c, d \leftarrow s_j, s_{(j+1) \bmod n}$ 
9:       if  $d(a, c) + d(b, d) < d(a, b) + d(c, d)$  then
10:         $\text{reverse } s_{i+1 \dots j}$ 
11:         $improved \leftarrow \text{true}$ 
12:        break out of both loops
13:      end if
14:    end for
15:  end for
16: end while
17: return  $s$ 

```

The improvement condition compares the cost of the two edges that would be removed, $d(a, b) + d(c, d)$, with the cost of the two edges that would replace them, $d(a, c) + d(b, d)$. Because the segment between the two cut points is simply reversed, no other edge weights change and the cost difference can be evaluated in constant time per candidate move [4].

2-Opt is used here both as a single solver and as the embedded *LocalSearch* of the Iterated Local Search algorithm described below.

B. Simulated Annealing

Plain local search like 2-Opt always stops at the first local minimum it reaches, because it never accepts a move that makes the tour worse. Simulated Annealing fixes this by sometimes accepting worse tours on purpose, so that the search can escape a local minimum and keep looking for a better solution [10].

How willing the algorithm is to accept a worse tour is controlled by a single number called *temperature* T . At each step a neighbor tour is created with one random 2-Opt move, and the change in tour length $\Delta = \text{Cost}(\text{next}) - \text{Cost}(\text{current})$ is computed. A better tour ($\Delta < 0$) is always accepted. A worse tour is accepted only with probability $e^{-\Delta/T}$. When T is high this probability is close to one, so many worse tours are accepted. When T is low it is close to zero, so the algorithm behaves almost like normal 2-Opt local search. The temperature is lowered after every step with a geometric cooling schedule, $T(t) = T_0 \cdot 0.9999^t$ [10].

To compare the algorithms in this study, the starting temperature T_0 is not fixed but estimated for each problem so that the algorithm fits the scale of the distances: a few hundred random 2-Opt moves are sampled, the average of the worsening moves $\bar{\Delta}$ (mean delta) is taken, and T_0 is set so that about 60% of such moves would be accepted at the start, i.e. $T_0 = \bar{\Delta} / \ln(1/0.6)$. The search starts from a greedy nearest-neighbor tour and runs for a fixed budget of 100 000 iterations

(or stops early once the temperature is practically zero), always returning the best tour seen so far.

Algorithm 2 Simulated Annealing

```

1:  $T_0 = \text{EstimateInitialTemperature}()$ 
2:  $current = \text{GreedyInitialTour}()$ 
3:  $best = current$ 
4: for  $t = 1$  to  $\text{MaxIterations}$  do
5:    $T = T_0 \cdot 0.9999^t$ 
6:   if  $T \approx 0$  then
7:     return  $best$ 
8:   end if
9:    $next = \text{Random2OptMove}(current)$ 
10:   $\Delta = \text{Cost}(next) - \text{Cost}(current)$ 
11:  if  $\Delta < 0$  then
12:     $current = next$ 
13:    if  $\text{Cost}(current) < \text{Cost}(best)$  then
14:       $best = current$ 
15:    end if
16:  else
17:    accept  $current = next$  with probability  $e^{-\Delta/T}$ 
18:  end if
19: end for
20: return  $best$ 

```

A random 2-Opt move is the same edge swap as in normal 2-Opt, but here the two cut points are chosen at random instead of by scanning for an improvement. This keeps each step cheap while still letting the temperature decide whether a worse tour is kept or thrown away.

C. Tabu Search

Tabu Search (TS) allows local search methods to overcome local optima. It does this by using a local search method (like 2-opt), but when it encounters a local minimum and can't find an improving move, it allows non-improving moves to escape it. To prevent the search from immediately undoing such a move and returning to the previous solution, it keeps a *tabu list* of recently performed moves. Moves that are in the tabu list are not allowed unless they satisfy an *aspiration criterion* (e.g., they lead to a new global best).

In algorithm 3, the search space is the set of all Hamiltonian tours. The neighborhood is defined by all 2-Opt swaps, giving $\mathcal{O}(n^2)$ candidate moves per iteration. The initial solution is constructed by the same greedy nearest-neighbor heuristic used for 2-Opt and Simulated Annealing.

At each iteration, the algorithm examines the whole 2-Opt neighborhood of the current tour. For every pair (i, j) , it generates a neighbor by reversing the segment $i + 1 \dots j$ and computes its total length. A move is considered admissible if either it is not tabu, or its tour length is better than the best known solution (aspiration criterion). Among all admissible moves, the one with the smallest tour length is selected (*best improvement* strategy).

The selected move is then applied to become the new current tour, and the move (represented as the pair (i, j)) is added to

the tabu list. If the tabu list exceeds a given length, the *tabu tenure*, the oldest entry is removed.

The general steps for tabu search are:

- 1) Construct initial solution s_0
- 2) While termination criterion not satisfied:
 - a) Find the tour s that minimizes the $\text{Cost}(s)$ from the current neighborhood that is either non-tabu or allowed by the aspiration criterion (e.g. all possible 2-Opt moves)
 - b) If that tour S is better than the current best $\text{Cost}(s) < \text{Cost}(s_{best})$ it becomes the new best tour.
 - c) Add the current move to the tabu list, replace oldest entry if it is full.
- 3) Return s_{best}

And how this is specifically implemented can be found in algorithm 3

Algorithm 3 Tabu Search

```

1:  $s \leftarrow \text{GreedyInitialTour}()$ 
2:  $s_{best} \leftarrow s$ 
3:  $\text{tabuList} \leftarrow \text{empty queue}$ 
4: for  $\text{iteration} = 1$  to  $\text{maxIterations}$  do
5:    $\text{bestNeighbor} \leftarrow \text{None}$ 
6:    $\text{bestMove} \leftarrow \text{None}$ 
7:   for each possible 2-Opt move  $(i, j)$  on  $s$  do
8:      $s' \leftarrow \text{tour after applying move } (i, j)$ 
9:      $\text{isTabu} \leftarrow (i, j) \in \text{tabuList}$ 
10:    if not  $\text{isTabu}$  or  $\text{Cost}(s') < \text{Cost}(s_{best})$  then
11:      if  $\text{Cost}(s') < \text{Cost}(\text{bestNeighbor})$  then
12:         $\text{bestNeighbor} \leftarrow s'$ 
13:         $\text{bestMove} \leftarrow (i, j)$ 
14:      end if
15:    end if
16:  end for
17:  if  $\text{bestNeighbor} = \text{None}$  then
18:    break
19:  end if
20:   $s \leftarrow \text{bestNeighbor}$ 
21:  add  $\text{bestMove}$  to  $\text{tabuList}$ 
22:  if  $\text{len}(\text{tabuList}) > \text{tenure}$  then
23:    remove oldest entry
24:  end if
25:  if  $\text{Cost}(s) < \text{Cost}(s_{best})$  then
26:     $s_{best} \leftarrow s$ 
27:  end if
28: end for
29: return  $s_{best}$  with closing edge

```

For our comparison, the tabu tenure is set to $\max(7, \lfloor \sqrt{n} \rfloor)$. This choice is based upon commonly used tabu tenure values $7, \sqrt{n}, n$ [11], [12].

The search continues for a fixed number of iterations, which depends on the problem size: $\text{maxIterations} = \max(50, 5000/n)$

This scaling ensures that large instances are not overwhelmed by an excessive number of iterations, while small instances still receive enough exploration. The algorithm returns the best tour found over the whole search.

Compared to Simulated Annealing, Tabu Search uses a deterministic move selection rule (best admissible neighbor) and relies on explicit memory to avoid cycles rather than on probabilistic acceptance. For the TSP, TS often finds very good solutions in a moderate number of iterations, because its best-improvement strategy aggressively descends toward local optima, while the tabu list provides just enough diversification to escape them. The implementation follows the classic template described in [13]

D. Iterated Local Search

The idea of iterated local search is that instead of searching the whole space of all candidate solutions, we focus only on candidate solutions in the local neighborhood of an initial solution. These are usually generated by some kind of local search heuristic. It is conceptually simple, but has led to state-of-the-art algorithms for many problems [6]. In its essence, iterated local search builds a chain of solutions by exploring the currently best solutions local neighborhood but introduces some randomness in order to not get stuck in local minima.

Algorithm 4 Iterated Local Search

```

1:  $s_0 = \text{GenerateInitialSolution}()$ 
2:  $s^* = \text{LocalSearch}(s_0)$ 
3:  $s_{\text{best}} = s^*$ 
4: repeat
5:    $s' = \text{Perturbation}(s^*)$ 
6:    $s^{*'} = \text{LocalSearch}(s')$ 
7:   if  $\text{Cost}(s^{*'}) < \text{Cost}(s_{\text{best}})$  then
8:      $s_{\text{best}} = s^{*'}$ 
9:   end if
10:  if  $\text{Cost}(s^{*'}) < \text{Cost}(s^*)$  then
11:     $s^* = s^{*'}$ 
12:  end if
13: until termination condition met (e.g., iterations = 200)
14: return  $s_{\text{best}}$ 

```

Iterated local search can be used with any heuristic optimization algorithm (*LocalSearch*) that takes in a tour and returns one that is at least as good as the input. It starts from some kind of initial solution s_0 that is improved to s^* by one iteration of the *LocalSearch* algorithm. This is the current solution. Then we try to improve s^* for a number of iterations or until a termination condition is met.

For the algorithm to be able to escape the local neighborhood, instead of applying *LocalSearch* directly to the current solution s^* again, first a change or *Perturbation* is applied that leads to an intermediate solution s' . Then *LocalSearch* is applied to s' and we get a new candidate solution $s^{*'}$. If this candidate solution $s^{*'}$ has a better *Cost* than the current one, it becomes the new current solution $s^{*'}$. If it is better than the current best, it also becomes the new current best s_{best} .

The *Cost* is the function that is to be minimized by the algorithm. In the case of the TSP that is the length of the tour.

The *Perturbation* is the mechanism that allows the algorithm to escape local minima. This function should not be reversible by the *LocalSearch*. So if the algorithm steps from $s_1^* \rightarrow s_2^*$ it then should be unable to do $s_2^* \rightarrow s_1^*$ in the same iteration.

Usual choices, and what was also used in this study are 2-Opt for the *LocalSearch* and 4-Opt for the *Perturbation*, this is a good choice because a 4-Opt move cannot be undone by a 2-Opt move and thereby return to the last starting point of the current iteration.

4-Opt (also called double-bridge move) randomly splits the tour into four segments $A + B + C + D$ and reconnects them as $A + C + B + D$ [6].

E. Genetic Algorithm

Genetic Algorithms (GAs) are population-based metaheuristics inspired by natural evolution [8]. A population of candidate tours evolves over several generations using selection, crossover, and mutation. The algorithm maintains a diverse set of solutions, which helps to explore the search space more broadly than local search alone.

1) *Representation and Initialization*: Each tour is represented as a permutation of the n city indices (without the closing edge). The initial population consists of $\text{POPULATION_SIZE} = 100$ individuals. One individual is created by the same greedy nearest-neighbor heuristic used for the other algorithms. The remaining 99 individuals are random permutations of the cities. This hybrid initialization ensures that a good starting solution is always present while providing diversity.

Algorithm 5 Genetic: Initialization of the Population

```

1:  $n \leftarrow$  number of cities
2:  $\text{popSize} \leftarrow 100$ 
3:  $\text{population} \leftarrow$  empty list
4:  $\text{greedy} \leftarrow \text{GreedyInitialTour}()$  {without closing edge}
5:  $\text{population.append}(\text{greedy})$ 
6: while  $\text{len}(\text{population}) < \text{popSize}$  do
7:    $\text{randomTour} \leftarrow$  random permutation of  $\{0, \dots, n-1\}$ 
8:    $\text{population.append}(\text{randomTour})$ 
9: end while
10: return  $\text{population}$ 

```

2) *Selection and Elitism*: At each generation, the next population is built as follows. First, the $\text{ELITISM_COUNT} = 2$ best tours of the current population are copied unchanged into the new population (*elitism*), guaranteeing that the best solution never degrades. The remaining slots are filled by repeatedly selecting two parents using *tournament selection*: Pick $\text{TOURNAMENT_SIZE} = 5$ random individuals from the population and return the one with the shortest tour length. This is done twice to obtain two parents. Tournament selection applies selection pressure without requiring global sorting.

Algorithm 6 Genetic: Tournament Selection

Require: population, costs, tournamentSize

- 1: indices $\leftarrow$ random choice of tournamentSize
 - 2: ↔indices from population
 - 3: bestIdx $\leftarrow$ index in indices with minimum cost
 - 4: **return** population[bestIdx]
-

3) *Crossover and Mutation:* With probability CROSSOVER_RATE = 0.9, the two parents are recombined using *Order Crossover (OX)*. OX works by:

- 1) choosing two random cut points $i < j$ in the parent permutation,
- 2) copying the segment between i and j from the first parent into the child at the same positions,
- 3) filling the remaining positions in the child with the cities from the second parent in the order they appear, skipping those already placed.

Algorithm 7 Genetic: Order Crossover (OX)

Require: parent1, parent2 (permutations of $0, \dots, n-1$)

- 1: $n \leftarrow \text{len}(\text{parent1})$
 - 2: $i, j \leftarrow$ uniformly random positions with $i \leq j$
 - 3: child $\leftarrow []$ with length n
 - 4: child[$i : j$] $\leftarrow$ parent1[$i : j$]
 - 5: fill $\leftarrow$ cities in parent2 not in child[$i : j$]
 - 6: $k \leftarrow n - j - 1$ {number of positions after j }
 - 7: child[$j + 1 : n$] $\leftarrow$ fill[$0 : k$] preserving cyclic order
 - 8: child[$0 : i - 1$] $\leftarrow$ fill[$k + 1 :$]
 - 9: **return** child
-

This operator preserves the relative order of cities from both parents and produces a valid tour. If crossover is not applied (with probability 0.1), the child is an exact copy of the first parent.

After crossover, each child is mutated with probability MUTATION_RATE = $1/n$. The mutation operator is *inversion*: Two random positions $i < j$ are chosen, and the segment between them is reversed. Inversion is a common mutation for permutation representations because it preserves the set of edges (i.e., does not introduce duplicate cities).

4) *Generations and Termination:* The number of generations is set to $\max(100, 10 \cdot n)$, where n is the number of cities. This scaling gives more generations to larger problems, allowing the GA to converge. After each generation, the best tour in the new population is compared with the global best found so far. The global best is updated if an improvement is found.

The algorithm returns the global best tour after the final generation.

Algorithm 8 Genetic: Inversion Mutation

Require: tour (list of cities)

- 1: $n \leftarrow \text{len}(\text{tour})$
 - 2: $i, j \leftarrow$ two distinct random positions in $[0, n-1]$, with $i \leq j$
 - 3: tour[$i : j + 1$] $\leftarrow$ reversed(tour[$i : j + 1$])
 - 4: **return** tour
-

The genetic algorithm does not use any local improvement. It relies entirely on crossover and mutation to evolve good tours. Despite this, the combination of a greedy initial solution, elitism, and the order crossover operator often yields competitive results, especially for smaller instances. For larger problems, the GA can be slower than local search methods, but it maintains a broad exploration of the search space.

Algorithm 9 Genetic Algorithm

- 1: population $\leftarrow$ InitializePopulation() {1 greedy + 99 random}
 - 2: $s_{\text{best}} \leftarrow$ best tour in population
 - 3: **for** generation = 1 **to** maxGenerations **do**
 - 4: newPopulation $\leftarrow$ elite individuals from population
 - 5: **while** newPopulation not full **do**
 - 6: $p_1 \leftarrow$ TournamentSelect(population)
 - 7: $p_2 \leftarrow$ TournamentSelect(population)
 - 8: **if** random float $\in [0, 1] < \text{crossoverRate}$ **then**
 - 9: child $\leftarrow$ OrderCrossover(p_1, p_2)
 - 10: **else**
 - 11: child $\leftarrow p_1$
 - 12: **end if**
 - 13: **if** random float $\in [0, 1] < \text{mutationRate}$ **then**
 - 14: child $\leftarrow$ InversionMutate(child)
 - 15: **end if**
 - 16: add child to newPopulation
 - 17: **end while**
 - 18: population $\leftarrow$ newPopulation
 - 19: $s_{\text{best}} \leftarrow \min(s_{\text{best}}, \text{best in population})$
 - 20: **end for**
 - 21: **return** s_{best} with closing edge
-

An advanced discussion of genetic algorithms for combinatorial optimization, including Gray-box optimization and the EAX crossover, is given in [8].

IV. SETUP

a) *Benchmark Instance Generation:* To compare the different algorithms presented in the previous section, a simple benchmark is with synthetic TSP instances is created.

For each problem size $n \in \{10, 25, 50, 100, 200, 500, 800, 1000, 1500, 2000\}$ 30 instances were generated per city distribution type.

The first set was created by sampling the cities coordinates uniformly from the unit square $[0, 1]^2$.

The second set was created to simulate clustered cities, this is more realistic, because due to terrain etc. cities are not distributed uniformly on the map. For each problem instance

- 1) the number of cluster is set depending on the problem size to $\max(1, n/10)$.
- 2) Cluster centers are sampled uniformly from $[0.1, 0.9]^2$ to avoid the borders.
- 3) Each subsequent node is drawn from a normal distribution around one of the cluster centers with $\sigma = 0.05$ until n is reached. An equal amount of nodes are drawn around each cluster center.

b) *Benchmark Ground Truths:* The optimal tours for the benchmark instances were obtained using the Concorde TSP solver. After computing the optimal tour, the tour length was recorded for later comparison.

c) *Baselines:* In addition to the above described algorithms and the ground truth computed by Concorde, we also provide a greedy nearest neighbor baseline for comparison. This is the most simple way to build a tour. Starting from some city, the next city to be visited is always the one that is closest to it, this is repeated until all cities have been visited.

d) *Hardware & Environment:* All experiments were conducted on a machine with a 32-core (64-thread) Intel Xeon Gold 6326 CPU, 504 GB of RAM.

V. RESULTS

In this chapter we present the results of the discussed algorithms in addition to the greedy baseline and Concorde on the benchmark instances. Figure 1 shows the optimality gap on the uniform instances along with the standard deviation across the 30 instances per size, figure 2. In general, the gap is higher on the clustered instances, than on the uniform sampled instances across all algorithms. However, this is mostly visible on the smaller sized instances with less than 500 nodes and their respective gaps converge on larger instances. One exception to that is simulated annealing, that performs worse on the uniform instances than on the clustered ones for size 200. Another one is tabu search on size 100 that also performs worse on the uniform instances.

Iterated local search consistently performs the best across all sizes and distributions, followed by tabu search and 2-opt, where tabu search dominates for instances smaller than 500 but drops in performance on instances bigger than that. They are followed with a big margin by simulated annealing and the genetic algorithm, where simulated annealing wins until size 200 but loses starting from size 500. Consistently the worst one is the greedy algorithm, although performance is pretty much the same as simulated annealing starting from size 500.

Interestingly, for most of the algorithms the gap is improving again after size 500, it plateaus there and then improves again.

When looking at the time scaling in figures 3, 4, it looks very similar between uniform and clustered instances again. There is a small difference between them for Concorde at 200 nodes and for tabu search at size 2000.

Greedy is the fastest by a big margin, followed by 2-opt for up to size 1000 before simulated annealing runs faster than it. After 2-opt follows the exact Concorde solver for most of the sizes, it is not directly comparable though because it

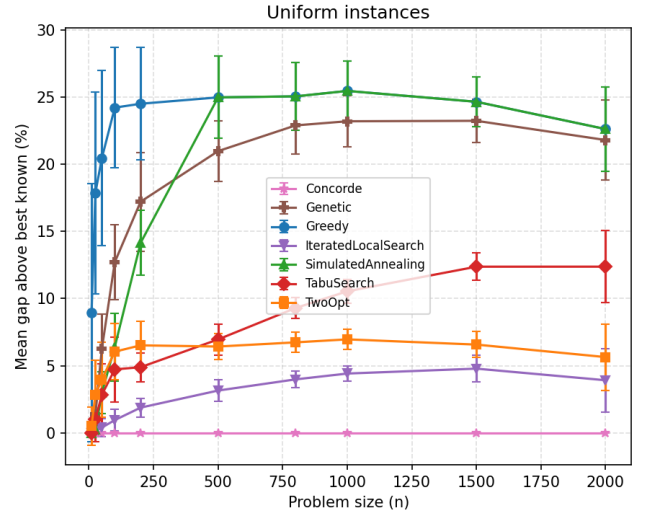


Fig. 1. Optimality gap of uniform algorithms for different sizes and distributions

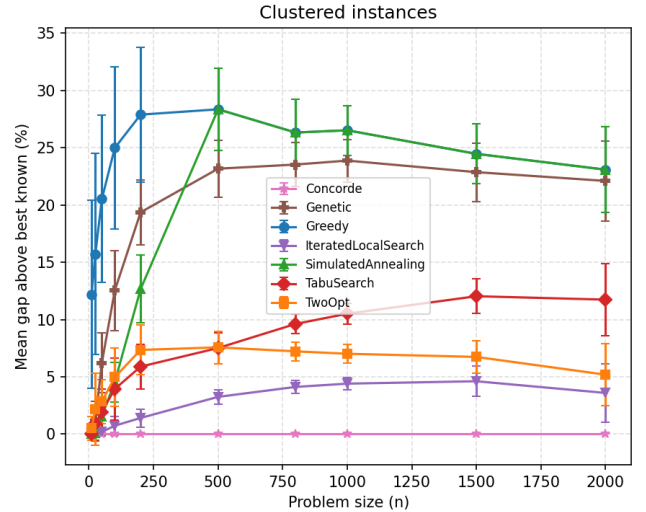


Fig. 2. Optimality gap of clustered algorithms for different sizes and distributions

was written in C instead of python. Afterwards iterated local search, tabu search, genetic algorithm and simulated annealing follow relatively close to each other, but they switch order after size 100.

Overall, although it also requires the most time for small instances, simulated annealing exhibits the best scaling behaviour, even running faster than Concorde and 2-opt for large instances.

VI. DISCUSSION

Trade off analysis. Speed vs Quality.

Best all rounder, fast but sloppy, accurate but expensive...

Explain why the results look like that. Why is one algorithm better able to handle clusters than another. etc.

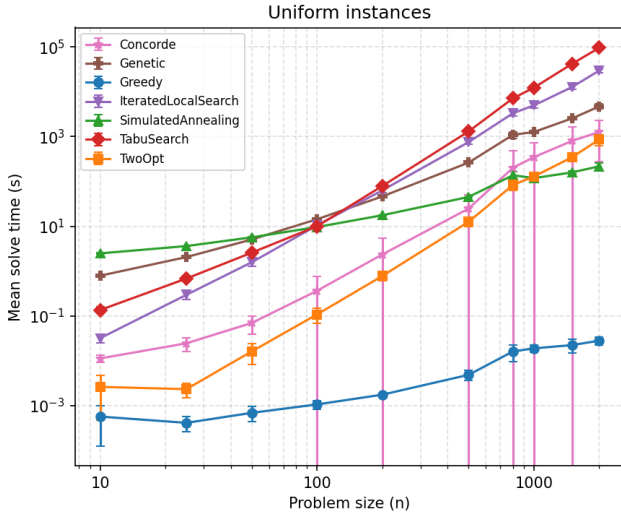


Fig. 3. Time scaling of uniform algorithms for different sizes and distributions.

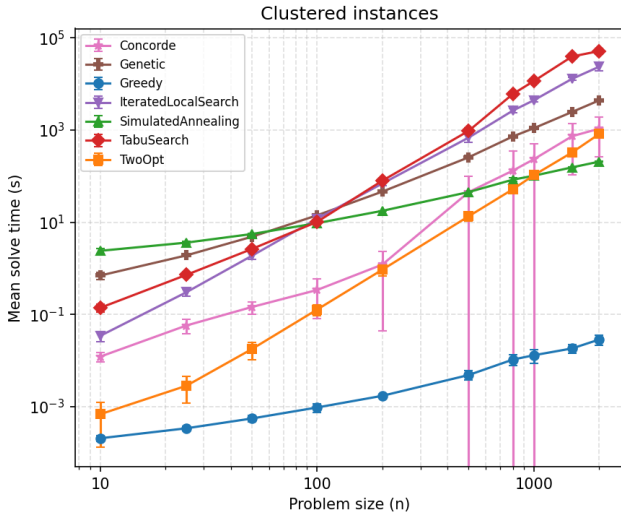


Fig. 4. Time scaling of clustered algorithms for different sizes and distributions.

Although the algorithms used to solve the benchmark problems were able to do so even for the largest instances, the time required to do so is really outside the bounds of practicality (ca. 27 hours for tabu search on clustered instances). Keep in mind though, that these are very basic version of these algorithms, and many extensions and more advanced versions exist.

A. Limitations

This study compares a small selection of existing algorithms available to solve the TSP, there are other approaches that solve it in a different way, however this selection of algorithm should give the reader a good overview of concepts and techniques to handle the NP-hard nature of this problem.

Most of these algorithms have some hyperparameters that could be tuned to achieve better results, potentially making this comparison unfair. The chosen values are sensible defaults but could be improved.

The performance of the algorithms is almost identical between uniform and clustered instances. This is unexpected because it is expected that some of the algorithms would be able to exploit the structure of these clusters to find better solutions faster, since solving a TSP with two distinct clusters essentially reduces to solving two separate smaller TSPs, reducing the overall problem size significantly. It is suspected that the generation of the clustered instances is possibly suboptimal. Since the number of clusters is set to $\max(1, n/10)$, there will be 10 nodes per cluster. This cluster size might be too small or the number of clusters too big for algorithms to effectively exploit these subgraph structures. Therefore the sampling approach for the clustered instances should be revised to produce more realistic graphs. Further experiments need to be done, but fall outside the scope of this study due to computation time constraints.

VII. CONCLUSION

We have presented a brief introduction to some of the most important methods to solve the Traveling Salesman Problem using heuristics and metaheuristics. After explaining the core concepts of these algorithms, we compared them using synthetic TSP instances, ranging in size from 10 to 2000 cities, and examined their scaling behaviour and their differences. This directly showcases them in a comparable way, providing new researchers in the TSP space with a good sense of how they perform. After reading this paper, it is our hope that the reader feels empowered to start their own research into TSP algorithms.

VIII. ACKNOWLEDGEMENT

The drafts of Sections III-C and III-E were translated with the assistance of DeepSeek [14] to improve the English language. The content was reviewed and corrected if the translation was not accurate to the content.

REFERENCES

- [1] M. R. Garey and D. S. Johnson, *Computers and Intractability*. wh freeman New York, 2002, vol. 29.
- [2] J. Sui, S. Ding, X. Huang, Y. Yu, R. Liu, B. Xia, Z. Ding, L. Xu, H. Zhang, C. Yu, and D. Bu, "A survey on deep learning-based algorithms for the traveling salesman problem," *Frontiers of Computer Science*, vol. 19, no. 6, p. 196322, Jun. 2025.
- [3] D. L. Applegate, R. E. Bixby, V. Chvátal, W. Cook, D. G. Espinoza, M. Goycoolea, and K. Helsgaun, "Certification of an optimal TSP tour through 85,900 cities," *Operations Research Letters*, vol. 37, no. 1, pp. 11–15, Jan. 2009.
- [4] C. Rego and F. Glover, "Local search and metaheuristics," in *The Traveling Salesman Problem and Its Variations*, G. Gutin and A. P. Punnen, Eds. Boston, MA: Springer US, 2007, pp. 309–368.
- [5] M. Gendreau and J.-Y. Potvin, "Tabu Search," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds. Cham: Springer International Publishing, 2019, pp. 37–55.
- [6] H. R. Lourenço, O. C. Martin, and T. Stützle, "Iterated Local Search: Framework and Applications," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds. Cham: Springer International Publishing, 2019, pp. 129–168.

- [7] P. Larrañaga, C. Kuijpers, R. Murga, I. Inza, and S. Dizdarevic, "Genetic Algorithms for the Travelling Salesman Problem: A Review of Representations and Operators," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 129–170, Apr. 1999.
- [8] D. Whitley, "Next Generation Genetic Algorithms: A User's Guide and Tutorial," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds. Cham: Springer International Publishing, 2019, pp. 245–274.
- [9] M. Gendreau and J.-Y. Potvin, Eds., *Handbook of Metaheuristics*, ser. International Series in Operations Research & Management Science. Cham: Springer International Publishing, 2019, vol. 272.
- [10] D. Delahaye, S. Chaimatanan, and M. Mongeau, "Simulated annealing: From basics to applications," in *Handbook of Metaheuristics*, M. Gendreau and J.-Y. Potvin, Eds. Cham: Springer International Publishing, 2019, pp. 1–35.
- [11] Krishnakumar R Iyer, "Tabu search algorithm for a thermal aware VLSI floorplanning application," 2013.
- [12] I. H. Osman, "Metastrategy simulated annealing and tabu search algorithms for the vehicle routing problem," *Annals of Operations Research*, vol. 41, no. 4, pp. 421–451, Dec. 1993.
- [13] E. K. Burke and G. Kendall, Eds., *Search Methodologies: Introductory Tutorials in Optimization and Decision Support Techniques*. Boston, MA: Springer US, 2014.
- [14] DeepSeek AI, "DeepSeek | ,," <https://www.chat.deepseek.com/>, 2025.